CHAPTER 15

CloudFormation and CDK

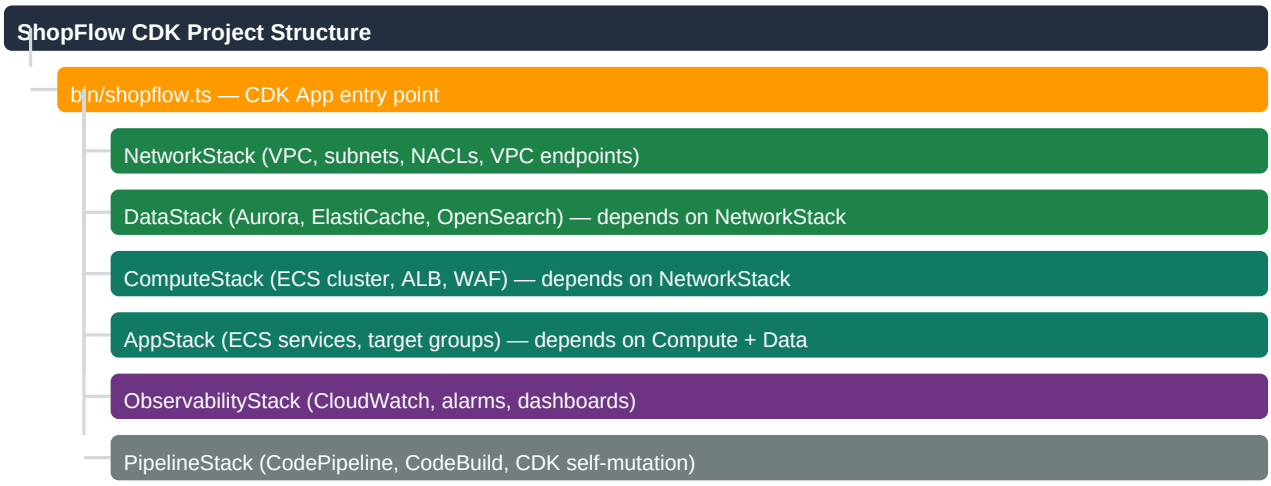
Constructs · Stacks · Pipelines · Drift Detection · Aspects · Testing

AWS Tutorial · ShopFlow

15-00 Chapter Overview — Infrastructure as Code at ShopFlow

ShopFlow uses AWS CDK (TypeScript) as the single source of truth for all infrastructure. Every AWS resource — VPC, ECS clusters, Aurora, ElastiCache, OpenSearch, S3, CloudFront, Route 53, WAF — is defined in CDK stacks. CDK synthesizes to CloudFormation templates; CloudFormation provisions the actual resources.

Layer	Tool	Role	ShopFlow Stack
IaC Language	CDK TypeScript	Define resources as typed classes	src/lib/*.ts
IaC Template	CloudFormation (CFn)	Provision/update AWS resources	Auto-generated by cdk synth
State Management	CloudFormation Stack	Tracks resource state; enables drift detection	shopflow-{env}-{component}
Deployment	cdk deploy via CodePipeline	CI/CD pipeline deploys on merge to main	ch16 covers CodePipeline
Secret Management	AWS Secrets Manager + SDK	DB passwords, API keys — never in CDK code	SSM Parameter Store for non-sensitive config



15-01 CDK Constructs — L1, L2, L3

CDK has three construct levels. L1 (Cfn*): direct CloudFormation resource wrappers; all properties available; verbose. L2: higher-level abstractions with defaults and convenience methods (new ec2.Vpc, new ecs.FargateService). L3: patterns composing multiple L2 constructs (ApplicationLoadBalancedFargateService). ShopFlow uses L2/L3 where available, L1 for unsupported resources.

Level	Class Prefix	Example	When to Use
L1 (CloudFormation Resource)	Cfn*	new ecs.CfnTaskDefinition(this,"td",{...})	No L2 equivalent; need all CFn prope
L2 (Intent-based)	(no prefix)	new ecs.FargateTaskDefinition(this,"td",{...})	Default choice — sensible defaults, ty
L3 (Pattern)	varies	new ecs_patterns.ApplicationLoadBalancedFargateService(...)	Quick complete setups; less customiz

L2 vs L3 — FargateService Example

```
// L3 (Pattern): creates ALB + ECS service + target group + listener rule
```

```
const svc = new ecs_patterns.ApplicationLoadBalancedFargateService(this,"Svc",{
cluster,
taskDefinition,
listenerPort: 443,
certificate: cert,
redirectHTTP: true,
});
// BUT: L3 creates its own ALB — cannot share with other services
// L2 (preferred for ShopFlow): full control, share existing ALB
const svc = new ecs.FargateService(this,"ProductSvc",{
cluster,
taskDefinition,
desiredCount: 3,
securityGroups: [sgProductSvc],
vpcSubnets: {subnetType: ec2.SubnetType.PRIVATE_WITH_EGRESS},
});
httpsListener.addTargets("ProductTg",{targets:[svc], port:8080});
```

15-02 CDK Stacks and Cross-Stack References

CDK splits infrastructure into multiple stacks for independent deployment and drift isolation. Cross-stack references share resources between stacks using Stack exports/imports. ShopFlow: NetworkStack exports the VPC; all other stacks import it. CloudFormation enforces: you cannot delete a stack export while another stack imports it.

Cross-Stack References CDK

```
// NetworkStack: export VPC
export class NetworkStack extends Stack {
public readonly vpc: ec2.Vpc;
constructor(scope: Construct, id: string, props: StackProps) {
super(scope, id, props);
this.vpc = new ec2.Vpc(this,"ShopFlowVpc",{
cidr: "10.0.0.0/16",
maxAzs: 3,
natGateways: 3,
subnetConfiguration: [
{name:"public", subnetType: ec2.SubnetType.PUBLIC, cidrMask:24},
{name:"private", subnetType: ec2.SubnetType.PRIVATE_WITH_EGRESS, cidrMask:24},
{name:"isolated", subnetType: ec2.SubnetType.PRIVATE_ISOLATED, cidrMask:24},
],
});
}
}
// DataStack: import VPC from NetworkStack
export class DataStack extends Stack {
constructor(scope: Construct, id: string,
props: StackProps & {vpc: ec2.Vpc}) {
super(scope, id, props);
const aurora = new rds.DatabaseCluster(this,"Aurora",{
engine: rds.DatabaseClusterEngine.auroraPostgres({
version: rds.AuroraPostgresEngineVersion.VER_15_4}),
vpc: props.vpc, // import VPC from NetworkStack
});
}
}
```

15-03 CDK Pipelines — Self-Mutating Pipeline

CDK Pipelines is a high-level construct that creates a CodePipeline pipeline that: (1) checks out CDK source code, (2) synthesizes CDK stacks to CloudFormation templates, (3) deploys stacks in the correct dependency order.

Self-mutating: the pipeline updates itself when the CDK pipeline definition changes before deploying application stacks.

CDK Pipeline CDK

```
const pipeline = new pipelines.CodePipeline(this,"ShopFlowPipeline",{
synth: new pipelines.ShellStep("Synth",{
input: pipelines.CodePipelineSource.gitHub(
"shopflow/shopflow-infra", "main",
{authentication: codestarConn}),
commands: [
"npm ci", // install CDK dependencies
"npm run build", // compile TypeScript
"npx cdk synth", // synthesize ALL stacks to cdk.out/
],
}),
dockerEnabledForSynth: true, // needed for Lambda bundling (esbuild)
crossAccountKeys: true, // encrypt pipeline artifacts with CMK
});
// Add deployment waves (parallel deploys within each wave)
const stagingWave = pipeline.addWave("Staging");
stagingWave.addStage(new
ShopFlowStage(this,"Staging",{env:{account:"111",region:"us-east-1"}}));
const prodWave = pipeline.addWave("Production",{
pre: [new pipelines.ManualApprovalStep("ApproveProduction")],
});
prodWave.addStage(new
ShopFlowStage(this,"Production",{env:{account:"222",region:"us-east-1"}}));
```

■ CDK Pipeline self-mutation: the first stage "UpdatePipeline" runs cdk deploy on the PipelineStack itself. If you add a new wave or change the pipeline structure in CDK code, the pipeline deploys the new pipeline definition first, then continues with the updated structure. This means pipeline changes take effect in the same pipeline run without manual intervention — the pipeline bootstraps itself.

15-04 CloudFormation Change Sets and Drift Detection

Before deploying, CloudFormation creates a Change Set showing exactly what will be added, modified, or deleted. ShopFlow CI/CD pipeline always creates a Change Set first, has a human review step for production deployments, and then executes the Change Set. Drift detection identifies manual AWS Console changes that deviate from the CDK definition.

Change Set and Drift Detection CDK

```
// Create change set (preview changes before deploying)
aws cloudformation create-change-set \
--stack-name shopflow-prod-data \
--change-set-name pre-deploy-preview \
--template-body file://cdk.out/DataStack.template.json \
--capabilities CAPABILITY_NAMED_IAM
// View change set (what will change):
aws cloudformation describe-change-set \
--stack-name shopflow-prod-data \
--change-set-name pre-deploy-preview \
--query "Changes[].ResourceChange.{Action:Action,Resource:ResourceType,Id:LogicalResourceId}"
// Detect drift (manual console changes):
aws cloudformation detect-stack-drift \
--stack-name shopflow-prod-data
// Response: lists resources with MODIFIED, DELETED, or NOT_CHECKED status
// Each drifted resource shows expected vs actual property values
```

Change Set Action	Meaning	Risk	ShopFlow Approval
Add	New resource will be created	Low (new resource)	Auto-approve
Modify (No Interruption)	Resource property update without restart	Low	Auto-approve
Modify (Some Interruption)	Resource restart required (e.g., ECS task redeploy)	Medium	Review
Modify (Replacement)	Resource deleted and recreated (new ARN/ID)	High — data loss risk	Manual approval required
Remove	Resource will be deleted	High — irreversible	Manual approval required

15-05 CDK Bootstrapping and Environments

CDK bootstrapping creates resources in each AWS account/region that CDK needs: an S3 bucket (stores synthesized CloudFormation templates), ECR repository (for Lambda Docker images), and IAM roles (for CDK deployments). Must be run once per account/region before first CDK deploy.

CDK Bootstrap Command

```
# Bootstrap staging account (111111111111) in us-east-1:
cdk bootstrap \
--profile staging \
aws://111111111111/us-east-1 \
--cloudformation-execution-policies arn:aws:iam::aws:policy/AdministratorAccess \
--trust 999999999999 # trust pipeline account to deploy to staging
# Bootstrap production account (222222222222):
cdk bootstrap \
--profile production \
aws://222222222222/us-east-1 \
--cloudformation-execution-policies arn:aws:iam::aws:policy/AdministratorAccess \
--trust 999999999999 # trust pipeline account
```

Bootstrap Resource	Name	Purpose
S3 Bucket	cdk-hnb659fds-assets-ACCOUNT-REGION	Stores CloudFormation template JSON and Lambda ZIP as
ECR Repository	cdk-hnb659fds-container-assets-ACCOUNT-REGION	Stores Docker images for Lambda container functions
CloudFormation Execution Role	cdk-hnb659fds-cfn-exec-role-ACCOUNT-REGION	CloudFormation assumes this role to create/update/delete r
Deploy Role	cdk-hnb659fds-deploy-role-ACCOUNT-REGION	CDK/CodePipeline assumes this to deploy stacks
Image Publishing Role	cdk-hnb659fds-image-publishing-role-ACCOUNT-REGION	CDK assumes this to push images to ECR bootstrap repo

15-06 CDK Aspects — Policy-as-Code

CDK Aspects visit every construct in the tree and can validate or modify properties. ShopFlow uses custom Aspects to enforce: (1) all S3 buckets have versioning enabled, (2) all ECS tasks have X-Ray daemon sidecar, (3) no security group allows 0.0.0.0/0 on port 22 (SSH), (4) all log groups have retention policy. Aspects run during cdk synth and can block deployment.

Custom Aspect — Enforce Log Retention CDK

```
// Aspect: enforce 30-day log retention on all LogGroups
class LogRetentionEnforcer implements IAspect {
visit(node: IConstruct): void {
if (node instanceof logs.LogGroup) {
const cfnLogGroup = node.node.defaultChild as logs.CfnLogGroup;
if (!cfnLogGroup.retentionInDays) {
// Enforce 30-day retention — never leave log groups with infinite retention
cfnLogGroup.retentionInDays = 30;
Annotations.of(node).addWarning(
"LogGroup had no retention; set to 30 days by LogRetentionEnforcer");
```

```
}
}
if (node instanceof s3.Bucket) {
// Block S3 buckets without versioning (required for compliance)
const cfnBucket = node.node.defaultChild as s3.CfnBucket;
if (cfnBucket.versioningConfiguration === undefined) {
Annotations.of(node).addError(
"S3 Bucket must have versioning enabled — compliance requirement");
}
}
}
}
// Apply to entire CDK app:
Aspects.of(app).add(new LogRetentionEnforcer());
```

15-07 CloudFormation Stack Policies and Deletion Protection

Stack policies prevent accidental changes to critical resources. ShopFlow applies a stack policy to production data stacks that: blocks any action on Aurora and ElastiCache resources (requires policy override to modify), and requires explicit confirmation for resource replacement. Deletion protection prevents the entire stack from being deleted.

Stack Policy and Deletion Protection CDK

```
// Enable termination protection on production stacks
const prodDataStack = new DataStack(app,"ProdData",{
env: {account: prodAccount, region: "us-east-1"},
terminationProtection: true, // cannot cdk destroy or delete from console
});
// Stack policy (applied via CLI — CDK does not yet support stack policies natively):
// aws cloudformation set-stack-policy --stack-name shopflow-prod-data \
// --stack-policy-body '{"Statement":[
// {"Effect":"Deny","Action":["Update:Replace","Update:Delete"],
// "Principal":"*","Resource":"LogicalResourceId/AuroraCluster"},
// {"Effect":"Deny","Action":["Update:Replace","Update:Delete"],
// "Principal":"*","Resource":"LogicalResourceId/RedisCluster"},
// {"Effect":"Allow","Action":"Update:*","Principal":"*","Resource":"*"}
// ]}'
```

Protection Level	Mechanism	Prevents	Override
Termination protection	terminationProtection: true	Stack deletion (cdk destroy, console dele	Disable via CLI/console before de
Stack policy (Deny Replace)	Stack policy JSON (UpdateReplace Deny)	Resource replacement (data loss risk)	Pass --override-policy during dep
S3 Object Lock (COMPLIANCE)	S3 bucket setting	Object deletion for retention period	Cannot be overridden even by ro
Aurora Deletion Protection	deletionProtection: true on cluster	Cluster deletion	Disable deletion protection first

15-08 CDK Testing — Unit and Integration Tests

CDK has a Testing module (aws-cdk-lib/assertions) for unit testing infrastructure code. ShopFlow CDK tests verify: VPC has 3 private subnets, ECS task definition has correct CPU/memory, ALB has TLS certificate, S3 bucket has versioning, all log groups have retention. Tests run in CI before cdk synth.

CDK Unit Tests

```
// cdk.test.ts (Jest)
import { Template } from "aws-cdk-lib/assertions";
import { App } from "aws-cdk-lib";
import { NetworkStack } from "../lib/network-stack";
describe("NetworkStack", () => {
const app = new App();
const stack = new NetworkStack(app,"TestNetwork",{
```

```
env: {account:"123456789",region:"us-east-1"}});
const template = Template.fromStack(stack);
test("VPC has 3 private subnets", () => {
template.resourceCountIs("AWS::EC2::Subnet",9); // 3 public + 3 private + 3 isolated
});
test("NAT Gateways in each AZ", () => {
template.resourceCountIs("AWS::EC2::NatGateway", 3);
});
test("VPC Flow Logs enabled", () => {
template.hasResourceProperties("AWS::EC2::FlowLog",{
TrafficType: "ALL",
LogDestinationType: "cloud-watch-logs",
});
});
});
```

■ Use Template.fromStack() for unit tests — synthesizes the stack in memory without an AWS account. Template.hasResourceProperties() asserts that at least one resource of the given type matches the partial properties. Template.resourceCountIs() counts resources. Use Jest snapshots (Template.toJSON()) to catch unexpected changes. Run npm test in CI before cdk synth to fail fast on broken infrastructure code.

15-09 CloudFormation Custom Resources

CloudFormation Custom Resources execute Lambda code during stack deployment to perform actions that CloudFormation does not natively support: seeding database initial data, rotating secrets immediately after creation, creating OpenSearch index mappings, and calling third-party APIs (register domain in external DNS provider).

Custom Resource — OpenSearch Index Initialization CDK

```
// Lambda: creates OpenSearch index with mapping during CDK deploy
const initIndexFn = new lambda.Function(this,"InitIndexFn",{
runtime: lambda.Runtime.NODEJS_20_X,
handler: "index.handler",
code: lambda.Code.fromAsset("./lambdas/init-os-index"),
vpc, timeout: Duration.minutes(5),
environment: {
OS_ENDPOINT: osDomain.domainEndpoint,
INDEX_NAME: "shopflow-products",
},
});
// Custom Resource: triggers initIndexFn on Create/Update
const initIndex = new cr.AwsCustomResource(this,"InitIndex",{
onCreate: {service:"Lambda", action:"invoke",
parameters: {FunctionName: initIndexFn.functionArn,
Payload: JSON.stringify({action:"create-index"})},
physicalResourceId: cr.PhysicalResourceId.of("init-os-index")},
policy: cr.AwsCustomResourcePolicy.fromSdkCalls({
resources: [initIndexFn.functionArn]}),
});
initIndex.node.addDependency(osDomain); // wait for OpenSearch to be ready
```

Custom Resource Use Case	Lambda Action	Timing	Error Handling
OpenSearch index initialization	PUT /index with mapping JSON	After osDomain create	Rollback stack if Lambda fails (non-20
Database schema seed	Run Flyway migrate via JDBC in VPC	After Aurora create	Stack rollback if seed fails
Secrets Manager rotation	Force immediate secret rotation	After DB create	Stack rollback if rotation fails
External DNS registration	API call to Cloudflare/Route53 external	After ALB create	Stack rollback; clean up DNS record

15-10 CDK Context and Environment Variables

CDK Context values (cdk.json or cdk context --context) configure per-environment differences without code changes. ShopFlow uses context for: Aurora instance class (db.r6g.large in prod, db.t3.medium in staging), ECS task minimum count, and feature flags (enableWaf: true/false). SSM Parameter Store resolves dynamic values at synth time.

CDK Context Usage

```
// cdk.json:
{
"context": {
"prod:auroraInstanceClass": "db.r6g.large",
"staging:auroraInstanceClass": "db.t3.medium",
"prod:ecsMinTaskCount": 3,
"staging:ecsMinTaskCount": 1,
"prod:enableWaf": true,
"staging:enableWaf": false
}
}
// DataStack.ts: read context value
const env = this.node.tryGetContext("env") as string; // "prod" or "staging"
const instanceClass = this.node.tryGetContext(`${env}:auroraInstanceClass`);
const cluster = new rds.DatabaseCluster(this,"Aurora",{
engine: rds.DatabaseClusterEngine.auroraPostgres({version:...}),
instanceProps: {
instanceType: new ec2.InstanceType(instanceClass), // db.r6g.large or db.t3.medium
vpc: props.vpc,
},
});
// Deploy with context: cdk deploy -c env=prod or -c env=staging
```

15-11 **CDK Nested Stacks for Large Templates**

CloudFormation has a limit of 500 resources per stack. ShopFlow AppStack has 180+ resources (ECS services, target groups, alarms, log groups). Nested Stacks split a parent stack into child stacks while keeping logical grouping. Unlike separate stacks, nested stacks are deployed together and cannot be independently deployed.

Nested Stack CDK

```
// Parent: AppStack
export class AppStack extends Stack {
constructor(scope: Construct, id: string, props: AppStackProps) {
super(scope, id, props);
// Nested stack: product-svc ECS service + alarms (separate CFn template)
const productNested = new ProductServiceNestedStack(this,"ProductSvc",{
vpc: props.vpc, cluster: props.cluster,
taskDef: props.productTaskDef,
});
// Nested stack: order-svc
const orderNested = new OrderServiceNestedStack(this,"OrderSvc",{...});
}
}
// class ProductServiceNestedStack extends NestedStack { ... }
// NestedStack synthesizes to a separate CFn template referenced by parent
// Parent stack deploys child stacks in parallel (no explicit dependency)
```

Property	Separate Stacks	Nested Stacks
Independent deployment	Yes (cdk deploy DataStack)	No — deployed with parent stack
Resource limit per unit	500 resources per stack	500 per nested stack (same limit)
Cross-stack references	Via CloudFormation Outputs/Imports	Via constructor props (no CFn exports)
Deletion	Must delete separately	Deleted with parent stack

Property	Separate Stacks	Nested Stacks
When to use	Independent lifecycle (data vs app)	Single lifecycle but > 500 resources

15-12 SSM Parameter Store for Configuration

SSM Parameter Store stores non-secret configuration values that vary per environment. ShopFlow uses SSM for: Aurora writer endpoint (set by CDK, read by ECS tasks at startup), OpenSearch domain endpoint, SQS queue URLs, and feature flags. Standard parameters are free; Advanced parameters (> 4KB, history) cost $0.05/parameter/month.

SSM Parameters CDK

```
// Write SSM parameters from CDK (during deploy):
new ssm.StringParameter(this,"AuroraEndpointParam",{
parameterName: "/shopflow/prod/aurora/writer-endpoint",
stringValue: auroraCluster.clusterEndpoint.hostname,
description: "Aurora PostgreSQL writer endpoint for product-svc",
tier: ssm.ParameterTier.STANDARD,
});
new ssm.StringParameter(this,"OsEndpointParam",{
parameterName: "/shopflow/prod/opensearch/endpoint",
stringValue: osDomain.domainEndpoint,
});
// Read SSM parameters in Spring Boot (at startup, not at request time):
// application.yaml:
spring:
datasource:
url: jdbc:postgresql://${aws.paramstore./shopflow/prod/aurora/writer-endpoint}:5432/shopflow
# Spring Cloud AWS SSM integration reads parameter at app startup
```

Config Type	Storage	ShopFlow Example	Access Pattern
Secrets (passwords, API keys)	Secrets Manager	DB password, Stripe API key	ECS task env var from secret; auto-rotation
Endpoints (Aurora, Redis, SQS)	SSM Parameter Store	aurora.endpoint, redis.endpoint	Spring Cloud AWS at startup (not CDK cros
Feature flags	SSM Parameter Store + AppConfig	/shopflow/prod/enableProductRecommendations=true	AppConfig for live changes without redeploy
Static config (region, env name)	CDK context + env var	AWS_REGION, SPRING_PROFILES_ACTIVE	ECS task definition environment map

15-13 CloudFormation Stack Sets for Multi-Account Deployment

ShopFlow uses separate AWS accounts for: pipeline (CodePipeline, CodeBuild), staging, and production. CloudFormation StackSets deploy the same stack to multiple accounts simultaneously. CDK Pipelines manages multi-account deployments automatically using cross-account deployment roles created during CDK bootstrap.

Account	Purpose	Stacks Deployed	CDK Environment
Pipeline account (999)	CodePipeline, CodeBuild, ECR	PipelineStack, SharedEcrStack	env: {account:999, region:us-east-1}
Staging account (111)	Full ShopFlow staging env	NetworkStack, DataStack, AppStack	env: {account:111, region:us-east-1}
Production account (222)	ShopFlow production	NetworkStack, DataStack, AppStack	env: {account:222, region:us-east-1}
DR account (333)	us-west-2 DR replica	NetworkStack, DataStack (replica)	env: {account:333, region:us-west-2}

Multi-Account CDK Pipeline

```
// bin/shopflow.ts — deploys to all accounts from pipeline account
const pipeline = new pipelines.CodePipeline(app,"Pipeline",{
synth: new pipelines.ShellStep("Synth",{
commands: ["npm ci","npm run build","npx cdk synth"],
}),
```

```
crossAccountKeys: true, // required for cross-account deployments
});
// Staging wave: pipeline account deploys to staging account (111)
pipeline.addStage(new ShopFlowStage(app,"Staging",{
env: {account:"111111111111",region:"us-east-1"}}));
// Production wave: with manual approval gate
pipeline.addStage(new ShopFlowStage(app,"Production",{
env: {account:"222222222222",region:"us-east-1"}}),{
pre: [new pipelines.ManualApprovalStep("ApproveProduction")],
});
```

15-14 CloudFormation Rollbacks and Stack Recovery

CloudFormation automatically rolls back a stack to its previous state if any resource update fails. Rollback: undo all changes in reverse order. If rollback itself fails (UPDATE_ROLLBACK_FAILED), the stack is stuck — must manually fix the failed resource before continuing rollback via the console or CLI.

Stack State	Meaning	Next Action
UPDATE_IN_PROGRESS	Stack update running	Wait
UPDATE_COMPLETE	All changes applied successfully	Normal operation
UPDATE_ROLLBACK_IN_PROGRESS	Update failed; rolling back changes	Wait for rollback to complete
UPDATE_ROLLBACK_COMPLETE	Rolled back to previous state	Fix issue in CDK code; redeploy
UPDATE_ROLLBACK_FAILED	Rollback itself failed (manual fix required)	Fix the specific resource manually; continue rollbac
DELETE_FAILED	Stack delete failed (resource still in use or protected)	Fix dependency; retry delete

Recover from UPDATE_ROLLBACK_FAILED

```
# Find the specific resource that caused rollback failure:
aws cloudformation describe-stack-events
--stack-name shopflow-prod-data
--query "StackEvents[?ResourceStatus=UPDATE_ROLLBACK_FAILED]"
# Option 1: Skip the failed resource (if safe to leave as-is):
aws cloudformation continue-update-rollback
--stack-name shopflow-prod-data
--resources-to-skip LogicalResourceId1
# Option 2: Manually fix the resource in the console,
# then retry: continue-update-rollback without --resources-to-skip
```

■ To prevent UPDATE_ROLLBACK_FAILED: enable rollback triggers on critical stacks. A rollback trigger is a CloudWatch alarm ARN — if the alarm fires during deployment, CloudFormation initiates rollback immediately (before the deployment completes). Set a latency alarm as a rollback trigger: if p99 latency exceeds 2s within 10 minutes of deployment start, auto-rollback. This prevents bad deployments from reaching COMPLETE state.

15-15 CDK Feature Flags and Removal Policies

CDK Feature Flags control behavior changes introduced across CDK major versions. ShopFlow pins feature flags in cdk.json to control: whether new S3 buckets are BlockPublicAccess by default, and behavior of VPC NAT gateway count. Removal policies control what happens to stateful resources when a CDK construct is removed from the stack.

Removal Policies for Stateful Resources

```
// S3 bucket: RETAIN (keep data even if stack is deleted)
const assetsBucket = new s3.Bucket(this,"Assets",{
removalPolicy: RemovalPolicy.RETAIN, // default for production S3
autoDeleteObjects: false, // do NOT auto-delete objects
});
// Aurora: SNAPSHOT (take final snapshot before deletion)
```

```
const aurora = new rds.DatabaseCluster(this,"Aurora",{
...
removalPolicy: RemovalPolicy.SNAPSHOT, // take snapshot on delete
deletionProtection: true, // belt AND suspenders
});
// CloudWatch Log Groups: RETAIN (keep logs for compliance)
const logGroup = new logs.LogGroup(this,"Logs",{
removalPolicy: RemovalPolicy.RETAIN,
retention: logs.RetentionDays.ONE_MONTH,
});
```

Resource	Removal Policy	Behavior on cdk destroy or stack delete
S3 bucket (assets)	RETAIN	Bucket and all objects kept; becomes orphan (no CFn management)
Aurora cluster	SNAPSHOT	Final snapshot created; cluster deleted after snapshot
CloudWatch Log Group	RETAIN	Log group and logs kept; unmanaged
ECS cluster (no state)	DESTROY	Cluster deleted immediately (safe — no data)
ElastiCache Redis	SNAPSHOT	Final snapshot; cluster deleted

15-16 CDK Escape Hatches for Advanced Customization

CDK escape hatches access the underlying CloudFormation (L1) resource from any L2 construct. Use when L2 does not expose a property you need. ShopFlow uses escape hatches to: add Aurora performance insights retention period (not in L2), configure OpenSearch UltraWarm (not in L2), and add custom CloudFormation resource metadata.

CDK Escape Hatch Examples

```
// Access L1 resource from L2 construct:
const aurora = new rds.DatabaseCluster(this,"Aurora",{...});
// Escape hatch: access CfnDBCluster (L1) from DatabaseCluster (L2)
const cfnAurora = aurora.node.defaultChild as rds.CfnDBCluster;
// Set Performance Insights retention (not exposed in L2 as of CDK 2.x):
cfnAurora.performanceInsightsRetentionPeriod = 7; // 7 days free tier
// Add DependsOn (L2 does not expose this for arbitrary resources):
cfnAurora.addDependency(cfnSecretAttachment);
// Override specific CloudFormation property:
cfnAurora.addPropertyOverride(
"MasterUserPassword",
"{{resolve:secretsmanager:shopflow/aurora/password:SecretString:password}}");
```

■ Prefer L2 construct properties over escape hatches — escape hatches bypass CDK type safety and may conflict with future L2 updates. If a property is critical and missing from L2, file a GitHub issue on aws/aws-cdk — L2 constructs are community-maintained. If you must use an escape hatch, add a TODO comment with the CDK issue number so it can be removed when L2 support arrives.

15-17 CloudFormation Macros and Transforms

CloudFormation Macros are Lambda functions that process CloudFormation templates during deployment. AWS::Serverless (SAM) Transform is the most common: transforms shorthand SAM syntax to full Lambda/API Gateway CloudFormation resources. ShopFlow uses the AWS::Include transform to include shared template fragments.

Transform	Type	What It Does	ShopFlow Usage
AWS::Serverless-2016-10-31	AWS-managed	SAM shorthand → full Lambda/API GW resources	Not used (CDK handles Lambda directly)
AWS::Include	AWS-managed	Includes a template fragment from S3 into the stack	Shared tagging policies from S3 fragmen
AWS::LanguageExtensions	AWS-managed	Dynamic references (Fn::ForEach, Fn::Length)	Complex loops in raw CFn (not needed w

Transform	Type	What It Does	ShopFlow Usage
Custom Macro (Lambda)	User-defined Lambda	Any template transformation logic	Policy enforcement at template level

AWS Include Transform for Shared Tags

```
// CloudFormation template: include shared resource tags from S3
{
"Transform": "AWS::Include",
"Parameters": {
"Location": {
"Fn::Sub": "s3://shopflow-cfn-fragments/tags/standard-tags.yaml"
}
}
}
// standard-tags.yaml (fragment):
// Tags:
// - Key: Project Value: ShopFlow
// - Key: ManagedBy Value: CDK
// - Key: CostCenter Value: engineering
// In CDK: Tags.of(app).add("Project","ShopFlow") is the preferred approach
```

15-18 CDK Hotswap Deployments for Development Speed

CDK Hotswap (--hotswap flag) deploys Lambda function code and ECS service task definition changes without a full CloudFormation stack update. Hotswap is 10-20x faster than full CFn deploy (seconds vs minutes). Use only for development/staging — hotswap skips CloudFormation state tracking (not suitable for production).

Deployment Method	Speed	CloudFormation State	Use In Production?
cdk deploy (full)	2-10 minutes	Full CFn update; drift tracking; rollback support	Yes — required
cdk deploy --hotswap	5-30 seconds	Bypasses CFn; no state tracking; no rollback	No — dev/staging only
cdk deploy --hotswap-fallback	5-30s (hotswap) or 2-10m (fallback)	Hotswap if possible; full CFn if not	No
cdk watch	Continuous (watches file changes)	Same as hotswap	No

CDK Watch for Rapid Iteration

```
# cdk.json: configure what to watch for hotswap
{
"watch": {
"include": [
"lambdas/**", // Lambda function code
"lib/**", // CDK stack changes
"*.ts" // TypeScript changes
],
"exclude": [
"**/*.d.ts",
"**/node_modules/**"
]
}
}
# Run: cdk watch --hotswap (in staging environment only)
# CDK detects file changes → hotswap deploys Lambda or ECS task in ~10s
# Developer workflow: edit Lambda code → save → hotswap → test in seconds
```

■ CDK watch + hotswap is ideal for Lambda development. The inner loop (edit → deploy → test) takes 10-15 seconds vs 3-5 minutes for full CFn. For ECS Fargate hotswap: CDK updates the task definition and triggers a new deployment (still ~2 minutes because ECS must start new tasks and drain old ones). Lambda hotswap is the biggest time saver: direct code update in seconds without touching CloudFormation.

15-19 Real-World CDK Anti-Patterns ShopFlow Fixed

ShopFlow encountered several CDK anti-patterns in early development. These are common mistakes that experienced CDK users learn over time. Each caused a production incident or deployment failure before being fixed.

Anti-Pattern	What Happened	Fix
Single monolith stack	Stack had 480 resources; 45-minute deploy even for small changes; rollback impossible	Split into 6 stacks: Network, Data, Compute, App, Observabi
No termination protection on prod	Developer ran cdk destroy prod-app by mistake; ECS services, RDS, and ALB deleted in 12 minutes	Enable terminationProtection: true on all production stacks; add CI/CD config ru
Hardcoded account IDs in CDK code	Deploying to a new AWS account failed because old account IDs were scattered throughout the codebase	Use Stack.of(this).account (CDK provides this); never hardcode account IDs
DESTROY removal policy on data stores	CDK stack update accidentally deleted Aurora cluster and all data; no snapshot existed	Explicitly set removalPolicy: RETAIN on all stateful resources from day
Secrets in CDK Context	DB password in cdk.json committed to git; visible to all developers	Use Secrets Manager; reference via secretsmanager.Secret

■ The Aurora DESTROY removal policy incident: a developer ran cdk deploy with a refactored stack that renamed the Aurora logical ID (from AuroraCluster to ShopFlowAuroraCluster). CloudFormation treated this as a delete-old + create-new. Since removalPolicy was DESTROY (the CDK default), Aurora was deleted without a snapshot. Data was recovered from the automated hourly backup (1-hour data loss). Fix: always explicitly set removalPolicy on all stateful resources in production CDK code. Do not rely on CDK defaults for production infrastructure.

15-20 CDK App Registry and Service Catalog

AWS Service Catalog Integration Registry (AWS AppRegistry) tracks which CloudFormation stacks belong to which application. ShopFlow registers all stacks (Network, Data, Compute, App, Observability) under the "ShopFlow" application. This enables cost allocation by application, compliance reporting across all stacks, and operational insights grouped by application.

AppRegistry CDK

```
const shopflowApp = new appreg.Application(this,"ShopFlowApp",{
applicationName: "ShopFlow",
description: "ShopFlow e-commerce platform — all production stacks",
});
// Associate all stacks with the application
[networkStack, dataStack, computeStack, appStack].forEach(stack => {
shopflowApp.associateStack(stack);
// Tags all resources in stack with: aws:cloudformation:application = arn:...
});
// Associate attribute group (metadata about the application)
const attrGroup = new appreg.AttributeGroup(this,"ShopFlowAttrs",{
attributeGroupName: "shopflow-app-metadata",
attributes: {
team: "platform-engineering",
costCenter: "CC-1234",
criticality: "tier-1",
oncallGroup: "shopflow-oncall",
documentation: "https://wiki.shopflow.internal/platform",
},
});
shopflowApp.associateAttributeGroup(attrGroup);
```

AppRegistry Benefit	How It Works	Business Value
Application cost view	AWS Cost Explorer: filter by application tag	See total ShopFlow cost across all stacks (not per-stack)
Resource inventory	List all resources in all associated stacks	Compliance: know every resource in the ShopFlow applicat
Compliance dashboard	AWS Config: rules evaluated per application	One-click compliance posture for entire ShopFlow platform
Operational insights	CloudWatch Application Insights: auto-detected application-level health without manual dashboard creation	

15-21 CDK Validation with cdk-nag

cdk-nag is an open-source CDK Aspect that runs security and compliance rules against CDK constructs at synth time. It implements AWS Security Hub, HIPAA, PCI-DSS, NIST, and SOC2 rule sets. ShopFlow uses the AwsSolutions rule set to catch: unencrypted S3 buckets, security groups with 0.0.0.0/0, Lambda without VPC, and IAM wildcards.

cdk-nag Integration

```
// Install: npm install cdk-nag
import { AwsSolutionsChecks, NagSuppressions } from "cdk-nag";
// Apply AwsSolutions checks to entire CDK app
Aspects.of(app).add(new AwsSolutionsChecks({ verbose: true }));
// Suppress known false positives with documented reason:
NagSuppressions.addStackSuppressions(appStack, [
{
id: "AwsSolutions-EC23",
reason: "ALB port 443 open to 0.0.0.0/0 — required for internet traffic",
},
{
id: "AwsSolutions-L1", // Lambda not using latest runtime
reason: "SearchIndexer Lambda pinned to nodejs18.x — tested and approved",
},
]);
// cdk synth fails if any non-suppressed cdk-nag rule fires
// CI pipeline: npm run synth must pass cdk-nag before deploy proceeds
```

cdk-nag Rule	What It Checks	ShopFlow Status
AwsSolutions-S1	S3 bucket server access logging enabled	Suppressed: access logs in separate bucket (cost)
AwsSolutions-RDS2	RDS encrypted at rest	Passes: storageEncrypted:true in CDK
AwsSolutions-IAM4	No AWS managed policy (use custom)	Partially suppressed: CodeBuild service role uses managed po
AwsSolutions-VPC7	VPC flow logs enabled	Passes: flowLogs enabled in NetworkStack
AwsSolutions-EC23	SG inbound from 0.0.0.0/0	Suppressed on ALB SG (required for internet access)

15-22 CDK Resource Tagging Strategy

All AWS resources must be tagged for cost allocation, ownership, and compliance. CDK Tags.of() applies tags recursively to all resources in a construct tree. ShopFlow tags every resource with: Project, Environment, ManagedBy, CostCenter, Team, and Criticality. AWS Config rule requires mandatory tags — non-compliant resources trigger alert.

CDK Resource Tagging

```
// bin/shopflow.ts: tag all resources in every stack
const app = new App();
// Tags applied to ALL constructs in entire CDK app
Tags.of(app).add("Project", "ShopFlow");
Tags.of(app).add("ManagedBy", "CDK");
Tags.of(app).add("CostCenter", "CC-1234");
Tags.of(app).add("Team", "platform-engineering");
// Per-stack tags (inherit app-level + add stack-specific):
Tags.of(prodDataStack).add("Environment", "production");
Tags.of(prodDataStack).add("Criticality", "tier-1");
Tags.of(prodDataStack).add("DataClassification", "confidential");
Tags.of(stagingDataStack).add("Environment", "staging");
Tags.of(stagingDataStack).add("Criticality", "tier-2");
```

Tag Key	Example Value	Purpose
Project	ShopFlow	Cost allocation: all ShopFlow costs in one Cost Explorer filter

Tag Key	Example Value	Purpose
Environment	production / staging	Separate prod vs staging costs; alert if staging costs exceed budget
ManagedBy	CDK	Identify resources created by CDK vs manual console (drift indicator)
CostCenter	CC-1234	Chargeback to business unit owning ShopFlow
Criticality	tier-1 / tier-2	Priority for incident response; tier-1 = wake on-call immediately
DataClassification	confidential / internal / public	Data governance; encryption enforcement

■ AWS Cost Explorer tag filtering requires tags to be activated as cost allocation tags in the Billing console. Tags are not automatically available for cost filtering — go to Billing → Cost Allocation Tags → Activate each tag key. After activation, tags appear in Cost Explorer within 24 hours. ShopFlow activates: Project, Environment, CostCenter. Filtering by Project=ShopFlow shows total ShopFlow infrastructure cost across all services and regions.

15-BP Best Practices and Anti-Patterns

✓ Split infrastructure into multiple stacks (Network, Data, Compute, App) — independent deployment, faster updates, smaller blast radius per change

✓ Use L2 constructs by default — sensible defaults, type-safe, reduces boilerplate; use L1 only when L2 does not expose a needed property

✓ Use CDK Aspects to enforce org-wide policies at synth time (log retention, encryption, versioning) — prevents policy violations from reaching deployment

✓ Enable terminationProtection on production stacks — prevents accidental stack deletion even with admin IAM permissions

✓ Always use Change Sets in production pipelines — review what will change before executing; block replacement actions requiring manual approval

✓ Use CDK unit tests (jest + aws-cdk-lib/assertions) — test infrastructure code like application code; catch VPC subnet count errors before deploy

✓ Store environment differences in CDK Context (cdk.json) — single codebase for staging and prod; no if/else for environment differences in stack code

✓ Bootstrap each account/region before first deploy — bootstrapping is idempotent; re-running is safe

✓ Use CDK Custom Resources for non-CloudFormation tasks (DB seed, OpenSearch index init) — keeps initialization in the IaC lifecycle

✓ Pin CDK library version in package.json — CDK breaking changes between minor versions; use exact version (not ^) in production

■ Never put secrets (DB passwords, API keys) in CDK code or cdk.json — use Secrets Manager; reference via secretsmanager.Secret.fromSecretNameV2()

■ Avoid circular cross-stack references — Stack A exports from Stack B which imports from Stack A. CDK detects this but the error message is confusing. Draw dependency diagram before splitting stacks

■ Do not use CfnOutput for sensitive values — CloudFormation Output values are visible in the console and CLI to anyone with cloudformation:DescribeStacks permission

■ Do not run cdk destroy on production stacks — even with termination protection disabled, this deletes all resources including stateful ones (Aurora, S3). Run destroy only on scratch environments

15-QR Quick Reference and Interview Q&A

Concept	Key Value / Rule
CDK vs Terraform	CDK: TypeScript/Python/Java (real programming language); native AWS constructs; L2 abstractions with
cdk synth vs cdk deploy	cdk synth: compile TypeScript → CloudFormation JSON template files (cdk.out/). No AWS changes. cdk d
Stack dependencies	props.vpc: ec2.Vpc passed as constructor parameter. CDK tracks this as a cross-stack reference: Networl

Concept	Key Value / Rule
Drift detection	CloudFormation detects drift by comparing expected template properties against actual resource properties
CDK Pipeline self-mutation	Step 1: UpdatePipeline stage deploys the PipelineStack itself (adds new stages, waves, or approval steps
Custom Resource lifecycle	CloudFormation calls Lambda with event.RequestType: Create (first deploy), Update (stack update), Delete